Django has a signaling system that allows you to react to certain events as they occur. This mechanism is designed as if it were meant for bigger things, but for the most part, it is used to interact with the ORM. In practice, I've only ever used it to do exactly what is described in this section: creating a second `Model` object as a side effect to the creation of a first.

Signals use the *observer/observable* pattern, meaning you register interest in an action and are notified when that action takes place. Figure 6.5 shows an overview of the model, signal registry, and signal handler interactions. Most ORM actions have two signals associated with each, a *pre signal* and a *post signal*. Pre signals are emitted just before the action is to take place, and post signals indicate the action has happened. It is possible to prevent an action inside a pre handler. For example, when saving an object, there is both a `pre_save` and `post_save` signal. If you register for the `pre_save` signal, the return value of your handler indicates whether you want the save to take place.

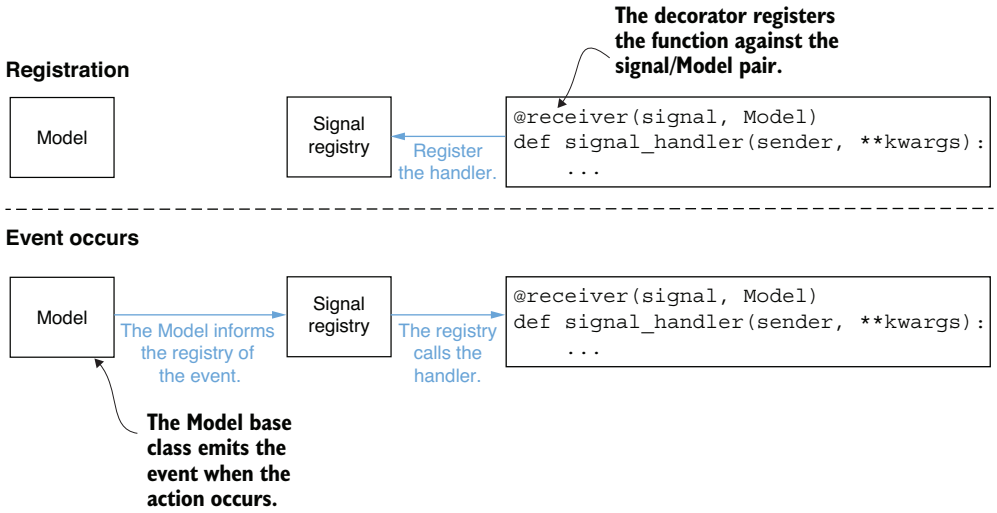


Figure 6.5 Registering and handling a signal from a Model

To create a `UserProfile` object as a side-effect to the `User` object being created, you trap the `post_save` signal, which is defined in `django.db.models.signals`. To register a signal handler, you use the `@receiver` decorator to wrap your handler function. The decorator connects a *sender* to a signal. The sender is the `Model` object whose actions you wish to receive.

There is no signal for a create action, only for save, which means your code needs to know the difference between a save for the first time and save for updating an object. To make it more complicated, there are also several ways to create a `Model` object: